

Mecha-Basketball 3D

Technical Report and User Manual

Alessandro Carotenuto

MSc Artificial Intelligence & Robotics, Sapienza University of Rome

July 11, 2026

Interactive Graphics — Prof. Marco Schaerf

Dept. of Computer, Control and Management Engineering (DIAG)

Repository <https://github.com/SapienzaInteractiveGraphicsCourse/final-project-intentional1>

Live demo [TODO: GitHub Pages link, once deployed \(see Section 12\)](#)

Contents

1	Project Overview	3
1.1	Robot Roster	3
1.2	Game Modes	3
2	Development Environment	3
3	Libraries, Tools, and External Assets	4
3.1	Third-Party Code	4
3.2	External 3D Assets	5
4	Hierarchical Models	5
4.1	The Shared 3R Arm	5
4.2	Per-Class Hierarchy	5
4.3	Robot Base Class and Stats	6
5	Materials, Lighting and Textures	6
5.1	Lights	6
5.2	Textures	7
5.3	Procedural Skybox	7
6	Animation	7
6.1	Ball Realism	7
7	Physics and Collisions	7
8	Audio	8
9	User Interaction and Controls	9
9.1	Main Menu Flow	9
9.2	Controls	9

9.3 Debug Tooling	9
10 1v1 Mode and Enemy AI	10
11 Point System	10
12 Known Limitations and Future Work	10
13 Conclusion	11

1 Project Overview

Mecha-Basketball 3D is a simulated basketball match between procedurally generated robots, built with Three.js and rendered on a GLTF basketball court. The user selects a robot class from a selection screen, each with a distinct stat profile and special move, and then plays or watches an animated simulation of the match — steering, dribbling, aiming and shooting are all driven by real-time user input rather than a scripted replay.

No external 3D model is used for the robots: each class is built entirely in Three.js from primitives, organized in hierarchical **THREE.Group** structures and animated procedurally in JavaScript. The only two external assets in the project are the basketball court and the basketball itself, both GLTF models (Section 3).

1.1 Robot Roster

Three robot classes are implemented, replacing an earlier concept (COLOSSUS/GLITCH/SENTINEL) that was scrapped before implementation began. All three are fully playable, both by the human player and by the enemy AI in 1v1 mode, and can be swapped from the Main Menu without a page reload.

Class	Locomotion	Special Move	Status
Mobile Manipulator (AMR)	Wheels	Dash	Implemented, playable
Legged Manipulator	Legs (real trot gait)	Jump	Implemented, playable
Drone	Flight (rotors)	Flight (grab-and-rise)	Implemented, playable

Table 1: Robot roster and status.

Each class shares a homogeneous stat struct { **speed**, **shooting**, **steal**, **block** } (same keys, different values), used both to drive gameplay (movement speed, shot-assist strength, steal/block cooldowns) and to render comparison bars on the selection screen.

1.2 Game Modes

Two game modes are selectable from the Main Menu:

- **Practice**: a single player controls one robot on an otherwise empty court, useful to test movement, dribbling and shooting without pressure.
- **1v1**: a second, AI-controlled robot is added (its class is chosen independently from the player’s), with tactical navigation, defense, and the ability to steal or block. The match ends when either side first reaches 11 points.

A third mode (3v3) was part of the original scope but was consciously dropped; see Section 12.

2 Development Environment

The project is built with **Three.js** (r0.178) on top of **Vite** (npm-installed, not loaded from a CDN), rather than raw WebGL. This choice trades low-level control of the GPU pipeline for a much smaller amount of boilerplate around shaders, buffers and transformation matrices, letting most of the development effort go into modelling, animation and interaction rather than into re-implementing what Three.js already provides — the same trade-off the course material itself makes explicit when it walks through the same rotating cube written first in plain WebGL and then in Three.js.

The renderer is configured for physically-plausible shading and soft shadows:

Tool	Role
Three.js ^0.178.0	Rendering engine (scene graph, cameras, lights, materials, post-processing)
Vite ^6.0.0	Dev server and bundler (<code>npm run dev</code> / <code>npm run build</code>)
Vanilla JavaScript (ES modules)	No TypeScript, no framework; native <code>import/export</code> throughout

Table 2: Core toolchain.

```

renderer.shadowMap.enabled = true
renderer.shadowMap.type = THREE.PCFSoftShadowMap // VSM was tried and is too
slow
renderer.toneMapping = THREE.ACESFilmicToneMapping
renderer.toneMappingExposure = 1.0
renderer.outputColorSpace = THREE.SRGBColorSpace

```

`vite.config.js` sets `base: './'` so the built bundle works when served from a GitHub Pages project subpath rather than a domain root.

3 Libraries, Tools, and External Assets

3.1 Third-Party Code

Beyond the Three.js core, the project uses only official Three.js *addons* (shipped with the `three` package itself, not separate dependencies) and no third-party animation or physics library.

Module	Purpose
<code>GLTFLoader</code>	Loads the court and ball models
<code>PointerLockControls</code>	Mouse-look camera in Spectate mode and for orbit input in Play mode
<code>EffectComposer</code> , <code>RenderPass</code> , <code>SSAOPass</code> , <code>OutputPass</code> , <code>SMAAPass</code>	Post-processing pipeline (ambient occlusion + anti-aliasing)
Sky (<code>three/addons/objects/Sky.js</code>)	Procedural atmospheric skybox (Preetham/Rayleigh scattering model), driving the Time-of-Day system

Table 3: Three.js addons used.

No physics engine (`Cannon.js`, `Ammo.js`, `Physijs`) was integrated. Collision response (Section 7) and the automatic dribble state machine are entirely hand-written, matching the project’s general preference for direct, imperative control over off-the-shelf simulation. No animation library (e.g. `tween.js`, suggested by the course material) is used either: every animation in the project — from a single joint rotation to the full shot trajectory — is computed by hand inside the render loop, satisfying the “no imported animations” requirement by construction rather than by omission.

3.2 External 3D Assets

Exactly two external, pre-made 3D assets are used, both GLTF models not authored by the team:

Asset	Notes
Basketball court	Sourced from Sketchfab, modified: a custom <code>GLTFLoader</code> plugin re-implements the <code>KHR_materials_pbrSpecularGlossiness</code> extension (removed from Three.js r152+ with no maintained replacement); several material colors were hand-edited in the raw <code>scene.gltf</code> to remove banding; court-line z-fighting fixed with <code>polygonOffset</code> .
Basketball	A dedicated GLTF (not the low-quality sphere baked into the court model), with color, normal and metalness/roughness maps — see Section 5.

Table 4: External 3D assets.

Every other visible object in the scene is procedurally generated: all three robot classes (geometry, materials and textures), the skybox, and all synthesized sound effects (Section 8). No robot model, texture image, or animation clip was downloaded or imported.

4 Hierarchical Models

4.1 The Shared 3R Arm

All three robot classes carry the same 3-revolute-joint manipulator arm on top of (or, for the Drone, hanging below) their locomotion base:

base (yaw) → link1 → elbow (pitch) → link2 (tapered) → wrist (pitch) → end effector + paddle

each joint being a nested `THREE.Group`, so that rotating a parent group (e.g. the elbow) carries every child (link2, wrist, paddle) with it — the textbook use of a scene-graph hierarchy for an articulated model. `link2` uses a tapered (frustum) cross-section built from a 4-sided `CylinderGeometry` rather than a uniform box, giving the forearm a visibly mechanical, non-uniform profile.

The arm code is deliberately *duplicated*, not shared through a common factory, across the three `ModelMaker` files (`AMRManipulatorModelMaker.js`, `LeggedManipulatorModelMaker.js`, `DroneModelMaker.js`): the arm is dense and delicate enough (kinematics, paddle convergence math, dribble tuning) that a premature shared abstraction was judged riskier than the duplication itself. What genuinely repeated *verbatim* across the three factories (geometry helpers, PBR material construction, scale/thickness setter boilerplate) was extracted into `geometryControlHelpers.js`.

4.2 Per-Class Hierarchy

- **Mobile Manipulator:** 4 wheels (tori) grouped under a disc chassis, with the arm mounted on top. Wheel group scale is derived from the disc radius so resizing the chassis keeps the wheels correctly attached.
- **Legged Manipulator:** the wheels are replaced by 4 two-joint legs (hip + knee, both pitch), positioned at the same four offsets the wheels would occupy, each oriented radially outward so the legs visibly radiate from the four corners instead of all bending the same way. The whole robot is scaled 25% larger than the Mobile Manipulator. A flat foot mesh is pivoted at its rear edge (ankle), not its center, mirroring the same off-center-pivot trick used for the paddle.

- **Drone**: a body with four rotor arms (propellers that spin continuously) replaces the wheeled chassis; the 3R arm is mounted *upside down*, hanging below the body through a fixed `rotation.x = \pi` flip group inserted between the yaw base and the first link.

4.3 Robot Base Class and Stats

A thin OOP layer, `RobotBase`, wraps each procedural factory: the constructor calls the passed-in factory and copies its returned fields (`root`, `joints`, `controls`, ...) onto the instance with `Object.assign`, so the geometry-construction code and the game-entity code stay separated. `RobotBase` owns the shared `RobotState` finite-state machine (`DRIBBLE` / `HANDLING` / `NO_BALL`), the `speed` getter (derived from the `speed` stat), and the default (empty) hooks for a special move, which each subclass overrides with its own behaviour.

Class	Speed	Shooting	Steal	Block
Mobile Manipulator	3	1	3	3
Legged Manipulator	2	3	2	5
Drone	5	2	1	1

Table 5: Stat values per class (scale 1–5, except Shooting which is 1–3).

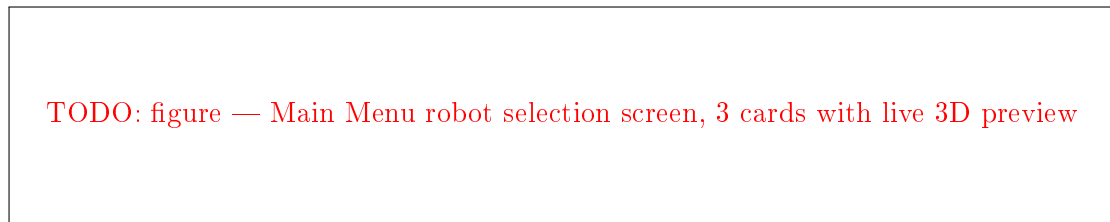


Figure 1: The robot selection screen, with live-dribbling 3D previews and stat bars.

5 Materials, Lighting and Textures

5.1 Lights

Light	Role
1 <code>HemisphereLight</code>	Warm ambient term approximating the image-based lighting the original Sketchfab scene used
1 <code>DirectionalLight</code>	Key light with a 4096 ² shadow map, frustum sized to cover the whole court
4 <code>PointLight</code>	Court lamp posts (candela-scale intensity, physically-based inverse-square falloff)
2 <code>SpotLight</code>	Hoop fixtures, lit only at Sunset/Night (toggled via intensity, never <code>.visible</code> , to avoid a shadow-map-allocation stutter the first time a light becomes visible)

Table 6: Scene lighting.

All light colors, intensities and the directional light’s position are driven by a `TimeOfDay` preset table (Sunrise/Day/Sunset/Night, selectable from the Main Menu), and cross-fade smoothly (2.5s, smoothstep-eased) between presets instead of snapping instantly — Section 6 details the transition system.

5.2 Textures

Two texture types are present, satisfying the “textures of at least two different kinds” requirement with margin:

- **Downloaded/external:** the basketball’s color map, and the court’s diffuse color map (from the Sketchfab source) — no normal/roughness map on the court geometry itself.
- **Procedurally generated:** color + normal + metalness/roughness maps on the basketball (dedicated GLTF); and normal + roughness + bump maps generated at runtime (height-field $\rightarrow$ gradient, the same technique used by comparable projects surveyed in the scouting phase) for every robot’s body/arm materials, for the wooden benches, and for the lamp-post material.

TODO: figure — close-up of a robot body showing procedural PBR texture detail

Figure 2: Procedurally generated normal/roughness detail on a robot’s chassis.

5.3 Procedural Skybox

The flat background color originally used behind the court was replaced with `three/addons/objects/Sky.js`, a procedural atmospheric-scattering skydome (Preetham model) — no HDR image is downloaded. The sky’s sun direction is always derived directly from the scene’s real `DirectionalLight` position (normalized), rather than tuned as an independent parameter, guaranteeing that the bright part of the sky and the direction shadows are actually cast from always agree. Night is a special case: the Preetham model is built for daylight scattering and degenerates visually below the horizon, so at Night the sky mesh is hidden and a flat dark color is used instead, with the swap happening only once the animated transition into Night has fully completed.

6 Animation

Every animation in the project is computed imperatively inside the render loop (`animate()`, driven by `requestAnimationFrame`); nothing is imported, and no `AnimationMixer`/keyframe clip is used anywhere.

6.1 Ball Realism

The ball’s rotation is derived every frame from its *real* frame-to-frame displacement (rolling axis = $\hat{u} \times \vec{v}$, angular speed = $|\vec{v}|/r$) rather than simulated separately for each of dribble/shot/handling/pickup, so spin is automatically consistent across every state the ball can be in.

7 Physics and Collisions

No physics engine is used (Section 3); every collision test is a hand-written closed-form formula:

- **Sphere vs. AABB** (ball vs. backboard/walls/poles/benches, and robot vs. static court geometry to keep a Dash from pushing a robot through a wall);
- **Sphere vs. torus** (ball vs. the rim), used both for physical bounce response and, separately, for exact-crossing detection when scoring;

System	Technique
Automatic dribble	A push/drop/rise finite-state machine, advanced at a fixed 120 Hz timestep (accumulator pattern, decoupled from the variable render <code>delta</code>) and synchronized with the elbow/link1 joint motion
Shot flight	Projectile motion under constant gravity, sub-stepped at 240 Hz to avoid tunnelling through the backboard at high shot speed; collision response against backboard/rim/walls/poles/benches
Wheel steering	Exponential-smoothing angular interpolation (shortest-path, wrap-safe) toward the current movement direction
Legged gait	A true diagonal-pair trot cycle, only advancing while the robot’s position is actually changing frame-to-frame
Drone flight feel	Body bank (turning) and thrust-tilt (accelerating), composed via explicit quaternion multiplication rather than raw Euler angles, to keep the tilt axis attached to the drone’s own nose regardless of its current yaw
Special moves	Dash (Mobile Manipulator, straight burst), Jump (Legged Manipulator, pure parabolic arc, gated to only work when not holding the ball), Flight (Drone, grab→rise→hold→descend, rigid ball attachment throughout)
Shot animation	Windup/release/recover arm pose, elbow coupled to camera pitch while aiming
Time-of-Day transitions	2.5 s smoothstep cross-fade of every light/sky parameter between presets, running every frame independently of game mode so it stays visible while chosen from the menu

Table 7: Animated systems (non-exhaustive; static geometry is limited to the court and lamp posts).

- both use the same reflection formula $\vec{v}' = \vec{v} - (1 + e)(\vec{v} \cdot \hat{n})\hat{n}$, with a different restitution coefficient e per material (backboard 0.15, rim 0.3, walls/poles 0.55, benches 0.5).

The 66 wall/bleacher collision boxes are extracted mesh-by-mesh from the real court GLTF (not a single aggregate bounding box), since the venue’s geometry has intentional gaps between panels that an aggregate box would either ignore or over-approximate.

A “hoop assist” potential field (position correction, not a velocity kick, to avoid overshooting the rim on slow close-range shots) gently pulls the ball toward the center of the rim while a shot is airborne near the basket, with strength scaled by the shooting robot’s `shooting` stat.

8 Audio

All sound effects are synthesized at runtime with the Web Audio API (oscillators and filtered white noise) — no audio file is downloaded or shipped with the project, mirroring the “no external assets” approach used for the robot textures.

A background music loop is not yet implemented; see Section 12.

Sound	Synthesis
Score	Two ascending sine-wave notes (a fifth apart)
Bounce	A low sine “body” tone sweeping downward, plus a short low-pass-filtered noise “contact” transient
Shoot	Band-pass filtered white noise, center frequency sweeping upward (a rising “whoosh”)
Steal	The same band-pass sweep technique, frequency sweeping <i>downward</i> and much shorter (a sharp “swipe”)
Block	Low-pass noise (a sharp hit) plus a descending low tone underneath

Table 8: Synthesized sound effects.

9 User Interaction and Controls

9.1 Main Menu Flow

The Main Menu (pure HTML/CSS overlaid on the canvas, no Three.js involved in the menu chrome itself) walks through: `GAMEMODES` (Practice/1v1) → `ROBOT` selection (3 cards, each with a live-dribbling 3D preview and stat bars; a second “Robot Opponent” screen appears only in 1v1) → `TIME OF DAY` (4 cards, animated preview behind the menu) with a `START` button on the same screen. All three robot classes are instantiated (hidden) at page load, so switching the selected class is only a visibility toggle and reference reassignment, never a page reload.

9.2 Controls

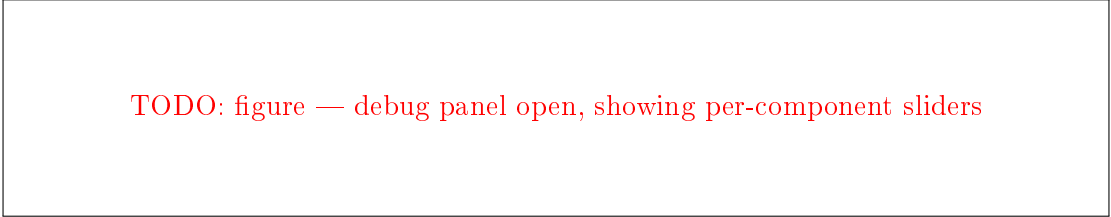
Input	Effect
<code>P</code>	Toggle the debug panel (per-component geometry tuning + live camera readout)
<code>M</code>	Toggle Spectate (free-fly) / Play (third-person) mode
Mouse (Play)	Orbits the camera around the robot; aims the arm
<code>WASD</code> (Play)	Move relative to camera facing; wheels/legs steer toward the movement direction
Left Shift (Play)	Special move for the active class (Dash / Jump / Flight)
Right mouse, held (Play)	<code>HANDLING</code> state — pauses the automatic dribble, frees the camera to aim toward the hoop
Left click (while <code>HANDLING</code>)	Shoot, direction from a raycast through the crosshair, constant force (halved inside the three-point arc)
Walk near a loose ball	Automatic pickup, no key required
<code>Q</code> (1v1, no ball)	Steal — a sweep toward the opponent
<code>E</code> (1v1, no ball)	Block — deflects a shot in flight
<code>Esc</code> (Play)	Pause menu
<code>1-8</code>	Toggle collision/contact wireframe overlays (backboard, rim, walls, poles, benches, steal zone, block radius, pickup zone)

Table 9: Full control scheme.

9.3 Debug Tooling

Beyond the gameplay controls, an in-page debug panel (key `P`) exposes every tunable parameter of the active robot’s geometry (link lengths/thicknesses/scales) and of the gameplay systems (dribble/shoot/handling timing constants) as live sliders, plus a “Copy Config” button that seri-

alizes the current geometry state to the clipboard. A separate “Animation Preview” section lets any pose (Dribble/Handling/Shoot/Special Move) be forced and inspected from a free Spectate camera angle, which is otherwise impossible for HANDLING (normally reachable only while holding the right mouse button, with the camera locked to the robot).



TODO: figure — debug panel open, showing per-component sliders

Figure 3: The debug panel, used to tune every robot’s geometry live.

10 1v1 Mode and Enemy AI

The opponent robot in 1v1 mode is driven by a small tactical finite-state machine, **EnemyState** $\in \{\text{CHASE_BALL}, \text{ATTACK}, \text{DEFEND}\}$, recomputed every frame from who currently possesses the ball (not sticky): it chases a loose ball, attacks toward its own hoop and shoots once in range when it has possession, and otherwise positions itself between the player and the player’s hoop while attempting a Steal or Block. Shot elevation for the AI is solved with the standard ballistic-projectile equation given a fixed launch speed, falling back to a 50° arc if the target is unreachable at that speed.

STEAL and BLOCK are shared between the human player and the AI through a single module (**CombatMoves.js**): a reach/sweep animation, a real bounding-box contact test (asymmetric for Steal — a wide margin in front of the sweeping robot, almost none behind it, so a Steal cannot succeed by walking up behind an opponent), and a resolve step that re-validates possession *at the end* of the sweep rather than at first contact.

Robot-vs-robot overlap is resolved with an asymmetric position correction based on ball possession: the robot currently holding the ball yields only 25% of the separation distance, the other robot yields 75% — “the one walking into someone gives way”, not a symmetric push-apart.

11 Point System

A shot is worth 2 points if the shooter’s position was inside the three-point arc *at the moment of release*, 3 otherwise; the arc radius is read directly from the court GLTF’s real accessor data rather than an estimated constant. Scoring itself is detected by interpolating the exact crossing point of the rim’s plane between two consecutive physics samples (not just testing the sample already past the plane), which matters most for steep, close-range shots where a single coarse sample can miss a shot that genuinely went in.

12 Known Limitations and Future Work

- **3v3 mode**: part of the original concept, never implemented; out of the final scope.
- **Background music**: only short synthesized sound effects exist (Section 8); a looping background track (menu and in-match, with a crossfade between the two) is planned but not yet implemented.
- **Robot color customization**: implemented (Main Menu $\rightarrow$ ROBOT $\rightarrow$ “Customize”, team-wide color for the player’s robot only), but intentionally does not persist across a

page refresh — it lives in an in-memory variable for the current session only.

- **TODO:** GitHub Pages deployment — `base: './'` is already configured in `vite.config.js`, but the project has not yet been deployed; the live link above needs to be filled in once it is, and the same link added to the repository's `README.md`.
- **TODO:** final polish pass and any last-minute animation tweaks, to be filled in closer to submission.
- **TODO:** personal conclusions / retrospective on the project, in the student's own words.

13 Conclusion

TODO: closing summary — once Sections 12's open items are resolved, summarize here what was learned/achieved, referencing back to the course's four mandatory requirements (hierarchical models, lights and textures, user interaction, animation) and how each is satisfied by the sections above.